

Resume Job Finder

2-Week Day-Wise Development Plan | AI/ML + FastAPI + Streamlit + Docker

PIPELINE READY	UI DONE	API DONE	DOCKER DONE	14 DAYS
----------------	---------	----------	-------------	---------

Project Overview

Goal	Build an AI-powered Resume Job Finder that extracts skills from uploaded PDFs and matches relevant jobs
Stack	FastAPI (Backend) Streamlit (UI) Docker + Docker Compose NLP / ML (Matching)
Starting Point	Basic pipeline built: PDF upload, text extraction, skill detection, document type detection
2-Week Target	Complete NLP matching, job data integration, scoring, UI polish, optimization & deployment-ready state

WEEK 1 — Core NLP, Matching Engine & Data Layer

Day 1	Project Audit & Architecture Planning <i>Focus: Solidify what's built, plan what's next</i> - Review existing code: <code>extract_text</code>, <code>clean_text</code>, <code>summarize</code>, <code>extract_skills</code>, <code>detect_type</code> - Define final feature list: skill extraction, job matching, scoring, result display - Sketch architecture diagram: PDF -> NLP -> Matcher -> Job DB -> UI - Set up Git branching strategy (main / dev / feature branches) - Write <code>requirements.txt</code> with all planned libraries PLANNING
Day 2	Improve Skill Extraction with spaCy / NER <i>Focus: Replace keyword matching with NLP-based extraction</i> - Install spaCy + <code>en_core_web_sm</code> model in Dockerfile - Replace simple SKILLS list matching with spaCy NER + <code>PhraseMatcher</code> - Build a richer SKILLS dictionary (50+ skills across categories) - Add skill categorization: Programming, Frameworks, Cloud, Soft Skills - Test on 3-4 sample resumes, measure extraction accuracy
Day 3	Resume Parsing — Sections & Entities <i>Focus: Extract structured info beyond just skills</i> - Parse resume sections: Experience, Education, Projects, Certifications - Extract candidate name, email, phone using regex + spaCy NER - Extract years of experience from date ranges in experience section - Store parsed data as a structured Python dict / JSON - Add <code>/parse</code> endpoint to FastAPI returning structured resume JSON API

Day 4	Job Data Layer — Static Dataset + Schema <i>Focus: Create job data to match against</i> • Create a JSON/CSV job dataset with 50+ realistic job postings • Each job: title, company, required_skills, experience_level, description • Design JobPosting schema (Pydantic model) in FastAPI • Build /jobs endpoint to list all available jobs • Add job filtering by skill or category via query params
Day 5	Matching Algorithm — Skill-Based Scoring <i>Focus: Core matching logic</i> • Implement cosine similarity or Jaccard score between resume skills & job skills • Score = (matched skills / total job skills) * 100 • Rank jobs by score, return top 10 matches • Add bonus scoring: experience match, education level match • Build /match endpoint: accepts resume text, returns ranked job list CORE FEATURE
Day 6	Semantic Matching with Sentence Transformers <i>Focus: Upgrade matching with embeddings</i> • Install sentence-transformers library • Encode resume summary + job descriptions into embeddings • Use cosine similarity on embeddings for semantic matching • Combine keyword score (40%) + semantic score (60%) for final score • Benchmark: compare keyword-only vs semantic results on test resumes
Day 7	Week 1 Integration & Testing <i>Focus: Wire everything together, fix bugs</i> • Connect all new endpoints: /parse, /match, /jobs to existing /upload • Full pipeline test: upload PDF -> parse -> match -> ranked results • Fix any Docker issues (new libraries in Dockerfile, rebuild images) • Write 5 unit tests for skill extraction and matching functions • Document all API endpoints in FastAPI's auto /docs MILESTONE

WEEK 2 — UI Enhancement, Optimization & Deployment Polish

Day
8

Streamlit UI — Results Dashboard

Focus: Display match results beautifully

- Show matched jobs as cards: title, company, match score (progress bar)
- Add skills matched vs missing display per job card
- Show extracted resume info: name, skills list, experience years
- Add color-coded score badges: green (>75%), yellow (50-75%), red (<50%)
- Add 'View Job Details' expander per card

UI

Day
9

Streamlit UI — Filters, Search & UX Polish

Focus: Make the UI user-friendly

- Add sidebar filters: min match score slider, job category dropdown
- Add search bar to filter jobs by keyword
- Add loading spinner during PDF processing
- Show upload progress and step-by-step processing status
- Add 'Download Results as CSV' button for matched jobs

Day
10

Performance Optimization — Backend

Focus: Make the API fast and efficient

- Add async file handling in FastAPI (already async, optimize further)
- Cache sentence-transformer model loading (load once at startup)
- Add response caching for repeated resume uploads (hash-based)
- Profile slow endpoints with Python cProfile or time logging
- Reduce Docker image size: use python:3.10-slim, clean pip cache

OPTIMIZATION

Day
11

Performance Optimization — NLP Pipeline

Focus: Speed up text processing

- Batch embed job descriptions at startup instead of per-request
- Pre-compute and store job embeddings as a .npy or pickle file
- Use spaCy's nlp.pipe() for batch processing instead of one-by-one
- Add PDF text extraction timeout (reject files > 10MB or > 20 pages)
- Test response time: target < 3 seconds for full match pipeline

Day
12

Error Handling, Validation & Security

Focus: Make the app robust

- Add file type validation: reject non-PDF uploads with clear error
- Add file size limit (configurable via env variable)
- Handle corrupted or empty PDF edge cases gracefully
- Add input sanitization for all API endpoints
- Return meaningful HTTP error codes (400, 422, 500) with messages

ROBUSTNESS

Day
13

Docker Optimization & Environment Config

Focus: Production-ready containers

- Add .env file support: load configs (model name, max file size, port) from env
- Add health check endpoint GET /health in FastAPI
- Update docker-compose.yml: add healthcheck, restart policy, volume mounts
- Add multi-stage Dockerfile for API to reduce final image size
- Test full docker compose up from scratch on a clean environment

Day 14

Final Testing, README & Demo Prep

Focus: Ship it

- End-to-end test with 5 different real resume PDFs
- Fix any remaining bugs found during testing
- Write a complete README.md: setup, usage, architecture diagram, screenshots
- Record a short demo (screen recording) of the full workflow
- Tag v1.0 release on Git, clean up code comments and dead code

COMPLETE

Technology Stack Reference

Layer	Technology	Purpose	When Added
Backend API	FastAPI + Uvicorn	REST API, file upload, endpoints	<i>Already built</i>
PDF Parsing	pdfplumber	Text extraction from PDF	<i>Already built</i>
NLP	spaCy + en_core_web_sm	NER, skill extraction, section parsing	<i>Day 2-3</i>
ML Matching	sentence-transformers	Semantic similarity matching	<i>Day 6</i>
Data	JSON job dataset	Static job postings for matching	<i>Day 4</i>
UI	Streamlit	Frontend, file upload, results display	<i>Already built</i>
Tokenization	NLTK punkt	Sentence tokenization for summary	<i>Already built</i>
Containers	Docker + Docker Compose	Isolated, reproducible environment	<i>Already built</i>
Optimization	NumPy, caching, async	Speed up NLP pipeline	<i>Day 10-11</i>
Config	.env + python-dotenv	Environment variable management	<i>Day 13</i>

Milestone Summary

Milestone	Day	Deliverable
NLP Skill Extraction Live	Day 2	spaCy-based skill extractor replaces keyword list
Structured Resume Parsing	Day 3	API returns name, email, skills, experience as JSON
Job Matching Engine Live	Day 5	/match endpoint returns scored & ranked job list
Semantic Matching Integrated	Day 6	Embedding-based matching for better accuracy
Week 1 Full Pipeline Working	Day 7	Upload PDF -> get ranked jobs end-to-end
UI Dashboard Complete	Day 9	Results cards, filters, CSV export in Streamlit
Optimized API (< 3s response)	Day 11	Caching, batched embeddings, profiled endpoints
Production-Ready Docker Setup	Day 13	Health checks, .env config, slim images
v1.0 Released	Day 14	Tested, documented, demo-ready project